

Parte 1 — Fundamentos: o pacote, os dados e a auditoria

Tutorial completo do sciModelFlowR (versão 1.0.2)

Walter Esfrain Pereira

2026-09-19

Índice

1	Como usar este tutorial	3
1.1	Instalação	4
2	O pacote em uma página	5
3	Dados simulados com discrepâncias plantadas	9
3.1	O ensaio de campo: outliers, ausências e heterocedasticidade	10
3.2	Dois desfechos: classe desbalanceada e contagem superdispersa	12
3.3	Estrutura: grupos, tempo, espaço e alta dimensão	14
4	Auditoria e contratos	19
4.1	Encontrando os outliers plantados	21
4.2	Contratos: o que o pacote recusa	22
5	Exercícios da Parte 1	24

1 Como usar este tutorial

Este material apresenta o `sciModelFlowR` do começo ao fim, na ordem em que um projeto científico real acontece: **perguntar, declarar, auditar, proteger, modelar, comparar, explicar, medir incerteza e relatar**. Ele foi escrito para quem nunca usou o pacote e quer aprender tanto a sintaxe quanto o raciocínio.

A diferença central em relação a um tutorial comum: **os dados são simulados com a verdade conhecida e discrepâncias plantadas de propósito** (outliers, ausências, heterocedasticidade, desbalanceamento, agrupamento, autocorrelação). Assim, cada método pode ser julgado contra o que foi plantado — e não contra uma expectativa vaga.

i Convenções deste material

- Todo bloco de código foi **executado** na versão 1.0.2; os números da prosa vêm do próprio código (nunca digitados à mão).
- Figuras e eixos usam rótulos em inglês (convenção científica); a explicação e as legendas estão em português.
- Números na prosa e nas tabelas usam **vírgula decimal**; os gráficos seguem o padrão do R (ponto), como é usual em figuras.
- Caixas coloridas indicam: **Interpretação** (como ler o resultado), **Armadilha** (erro comum), **Boa prática**, **No seu experimento** (transposição para dados reais) e **Exercício**.

O tutorial tem seis partes:

Parte	Tema	Módulos do pacote
1	Fundamentos: o pacote, os dados e a auditoria	<code>doctor</code> , <code>datasets</code> , <code>data-audit</code> , <code>design</code>
2	Desenho, reamostragem e preparação	<code>resampling</code> , <code>bootstrap</code> , <code>preprocess</code> , <code>features</code> , <code>imbalance</code>
3	Modelagem supervisionada e probabilidades	<code>model-adapters</code> , <code>experiment</code> , <code>diagnostics</code> , <code>probabilistic</code>
4	Tuning, benchmark, calibração e limiares	<code>tuning</code> , <code>benchmark</code> , <code>calibration</code>
5	Explicabilidade e incerteza	<code>explain</code> , <code>conformal</code> , <code>uncertainty</code>
6	Avançado e produção: Bayes, DL, persistência e relato	<code>bayesian</code> , <code>deep-learning</code> , <code>tracking</code> , <code>persistence</code> , <code>reporting</code>

1.1 Instalação

```
# Instala a versao publicada, com as vinhetas:  
remotes::install_github("wep69/sciModelFlowR", build_vignettes = TRUE)  
library(sciModelFlowR)
```

2 O pacote em uma página

O `sciModelFlowR` organiza um projeto de modelagem como um **contrato declarado**: tarefa, dados, desenho, pré-processamento, reamostragem, modelo e métricas são objetos explícitos. A partir daí, o pacote **recusa** atalhos que produziriam números otimistas (por exemplo, pré-processar antes de dividir ou ajustar hiperparâmetros no teste). O diagnóstico do ambiente é o primeiro comando de qualquer sessão.

```
doutor <- smf_doctor()
cat("componentes verificados:", nrow(doutor),
    "| disponíveis:", sum(doutor$available), "\n")
```

componentes verificados: 31 | disponíveis: 31

```
tab_smf(subset(doutor, component %in% c("R", "sciModelFlowR", "stats", "torch", "xgboost", "brms"),
    caption = "Diagnóstico do ambiente: componentes centrais e backends opcionais.")
```

Tabela 2.1: Diagnóstico do ambiente: componentes centrais e backends opcionais.

	component	available	version	status
1	R	TRUE	4.6.0	core
2	sciModelFlowR	TRUE	1.0.2	core
13	xgboost	TRUE	3.2.1.1	optional-available
16	torch	TRUE	0.17.0	optional-available
19	brms	TRUE	2.23.0	optional-available

O pacote distribui 25 conjuntos sintéticos de validação (“gold”), úteis para estudar o comportamento esperado antes de usar dados próprios. Cada um tem uma **carta** que descreve cenário, variáveis, verdade conhecida e armadilhas pedagógicas.

```
cartao <- smf_dataset_card("gold_linear_regression")
cat(substr(cartao, 1, 480), "...\\n")
```

```
# gold_linear_regression
```

```
## Scientific scenario
```

Synthetic agronomic yield response with independent observational units and a Gaussian linear d generating process. The dataset is intended to validate schema handling, holdout splitting, tra only preprocessing, reference linear-model adapters, prediction metrics, serialization, and rep

```
## Variables
```

- `obs\_id`: stable observation identifier.
- `nitrogen`: nitrogen dose, arbitrary teaching units ...

A API pública tem 246 funções, organizadas em 10 categorias. O mapa abaixo é a melhor forma de saber “onde mora” cada assunto.

```
api <- smf_api_catalog()
por_categoria <- as.data.frame(table(api$category), stringsAsFactors = FALSE)
names(por_categoria) <- c("categoria", "funcoes")
por_categoria <- por_categoria[order(-por_categoria$funcoes), ]
tab_smf(por_categoria, caption = "Funções exportadas por categoria da API (1.0.2).")
```

Tabela 2.2: Funções exportadas por categoria da API (1.0.2).

	categoria	funcoes
1	core	88
6	modeling	35
5	inference-uncertainty	24
3	deep-learning	23
9	resampling-features	23
7	operations	17
8	reporting-validation	11
10	tuning-benchmark	10
2	data-design	9
4	explainability	6

```
ggplot(por_categoria, aes(stats::reorder(categoria, funcoes), funcoes)) +
  geom_col(fill = cores_smf[["primaria"]], width = .7) +
  geom_text(aes(label = funcoes), hjust = -0.2, size = 3.2) +
  coord_flip() +
  labs(x = NULL, y = "funções exportadas",
       title = "API map: 246 exports by category") +
  ylim(0, max(por_categoria$funcoes) * 1.12)
```

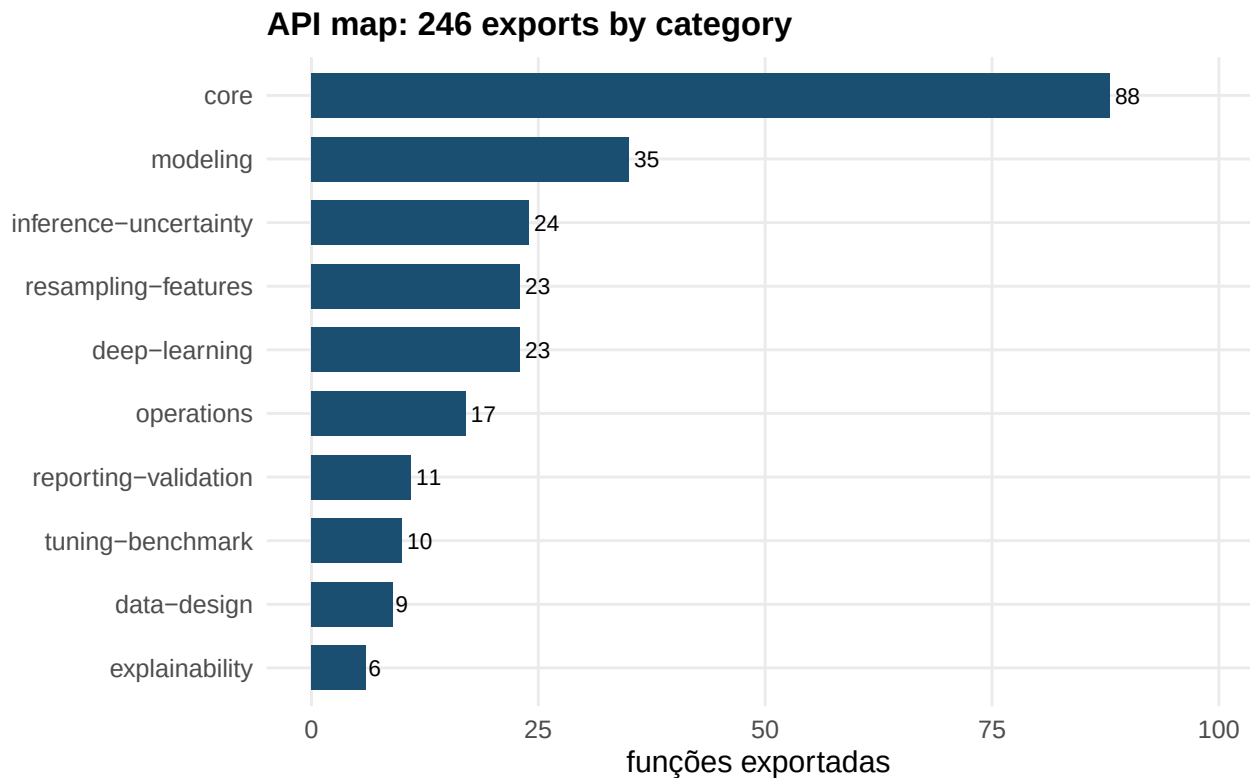


Figura 2.1: Distribuição das 246 funções exportadas por categoria.

Os backends opcionais são declarados, não presumidos: a matriz abaixo diz o que é núcleo e o que depende de pacote externo, e o diagnóstico mostra o que está disponível **neste** computador.

```
matriz <- smf_backend_matrix()
tab_smf(matriz[, c("backend", "tier", "package", "scope")],
        caption = "Matriz de backends: núcleo e opcionais.")
```

Tabela 2.3: Matriz de backends: núcleo e opcionais.

backend	tier	package	scope
stats	core-reference	base R	lm/glm differential reference
tidymodels	optional	parsnip/workflows	supervised modeling
mlr3	optional	mlr3	supervised modeling
xgboost	optional	xgboost	tree boosting
torch	optional	torch	Deep Learning
luz	optional	luz	DL trainer
keras3	optional	keras3	DL alternate backend
brms	optional	brms/cmdstanr	Bayesian multilevel/distributional
dbarts	optional	dbarts	BART
DiceKriging	optional	DiceKriging	Gaussian process
probably	optional	probably	calibration/conformal
mlflow	optional	mlflow	tracking

backend	tier	package	scope
pins	optional	pins	artifact registry
vetiver	optional	vetiver	deployment

```

estado <- smf_capabilities()
resumo_back <- data.frame(
  backend = names(estado),
  status = vapply(estado, function(x) x$status, character(1))
)
tab_smf(resumo_back, caption = "Estado de cada backend no ambiente atual.")

```

Tabela 2.4: Estado de cada backend no ambiente atual.

	backend	status
brms	brms	advanced
dbarts	dbarts	advanced
DiceKriging	DiceKriging	advanced
keras3	keras3	advanced
luz	luz	advanced
mlr3	mlr3	advanced
probably	probably	advanced
stats	stats	core
tidymodels	tidymodels	advanced
torch	torch	advanced
xgboost	xgboost	advanced

💡 Boa prática

Rode `smf_doctor()` no início de cada sessão e guarde a saída no relatório. “Funcionou na minha máquina” deixa de ser desculpa quando o diagnóstico fica registrado junto com o resultado.

3 Dados simulados com discrepâncias plantadas

Nove conjuntos foram simulados para cobrir todo o ciclo de modelagem. A tabela é o **gabarito**: diz o que foi plantado e qual é a verdade. Cada parte do tutorial usa um subconjunto e compara o que o método recuperou com o que está aqui.

```
gab <- gabarito
gab$detalhe <- substr(gab$detalhe, 1, 78)
names(gab) <- c("Conjunto", "n", "p", "Discrepância", "O que foi plantado", "Verdade")
tab_smf(gab[, c("Conjunto", "n", "p", "Discrepância", "O que foi plantado")],
        caption = "Gabarito dos nove conjuntos simulados (detalhe completo no arquivo dados/gaba-
                    rito.csv).")
```

Tabela 3.1: Gabarito dos nove conjuntos simulados (detalhe completo no arquivo dados/gabarito.csv).

Conjunto	n	p	Discrepância	O que foi plantado
field_yield	180	8	outliers	5 parcelas (P023, P067, P101, P145, P177) deslocadas entre -3000 e +4100
field_yield	180	8	ausencias	11 NA em nitrogenio (MCAR 6%); 16 em organic_matter (MAR: bloco B4)
field_yield	180	8	heterocedasticidade	sigma do erro = 100 + 1.5 * nitrogenio
field_yield	180	8	efeito de bloco	efeitos B2=+120, B3=-80, B4=+60 kg/ha
discase_binary	400	6	desbalanceamento	10.5% de casos positivos
discase_binary	400	6	ruido de rotulo	20 plantas com rotulo invertido (5%)
farm_groups	240	4	agrupamento	24 fazendas, 10 talhoes cada; intercepto aleatorio sd=300
climate_series	730	4	choques	5 dias com deslocamento de +5,5 a +7,2 graus
soil_spatial	200	4	autocorrelacao espacial	covariancia exponencial: patamar 0,6; alcance 0,25; pepita 0,15
spectral_pn	120	202	alta dimensao	200 comprimentos de onda para 120 amostras; multicolinearidade severa
spectral_pn	120	202	outliers espectrais	4 espectros (A011, A047, A083, A109) deslocados em +0,35
pest_counts	150	4	superdispersao	binomial negativa com theta = 2 (variancia > media)
pest_counts	150	4	zeros extras	22 zeros plantados alem do modelo (15%)
dl_tiny	300	4	nao linearidade	superficie com seno, quadrado e interacao; sem outlier

Conjunto	n	p	Discrepância	O que foi plantado
bayes_small	30	2	amostra pequena	n = 30; prior informativo conhecido para o conjugado

3.1 O ensaio de campo: outliers, ausências e heterocedasticidade

```
campo <- ler("field_yield")
campo$origem <- ifelse(campo$plot_id %in% verdade$field_yield$outlier_ids,
                      "outlier plantado", "parcela regular")
cat("parcelas:", nrow(campo),
    "| NA em nitrogen:", sum(is.na(campo$nitrogen)),
    "| NA em organic_matter:", sum(is.na(campo$organic_matter)), "\n")
```

parcelas: 180 | NA em nitrogen: 11 | NA em organic_matter: 16

```
ggplot(campo, aes(nitrogen, yield)) +
  geom_point(aes(colour = origem), size = 2.2, alpha = .85) +
  scale_colour_manual(values = c("parcela regular" = cores_smf[["primaria"]],
                                "outlier plantado" = cores_smf[["alerta"]])) +
  labs(x = "Nitrogen (kg/ha)", y = "Yield (kg/ha)", colour = NULL,
       title = "Field trial: planted outliers") +
  theme(legend.position = "top")
```

Field trial: planted outliers

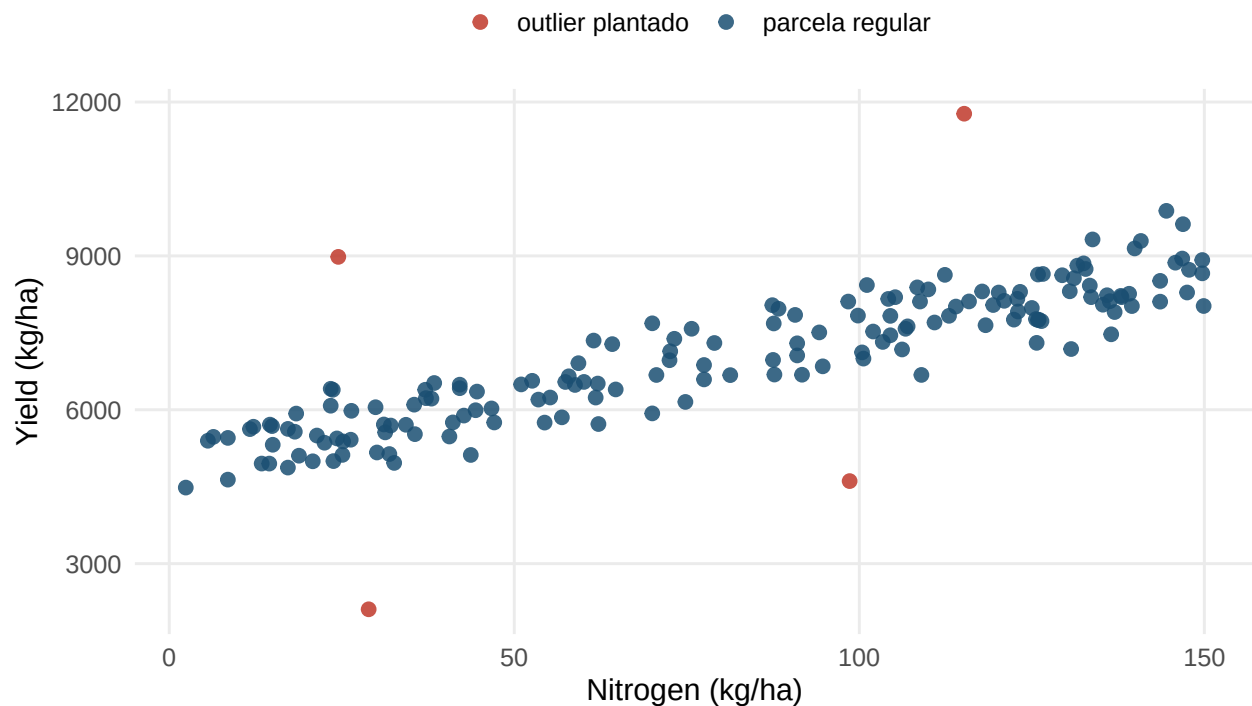


Figura 3.1: Ensaio de campo simulado: rendimento em função do nitrogênio, com os cinco outliers plantados destacados.

i Interpretação

A nuvem mostra a relação esperada (mais nitrogênio, mais rendimento) e também que a **dispersão cresce** com o nitrogênio: é a heterocedasticidade plantada ($100 + 1.5 * \text{nitrogen}$). Os cinco pontos vermelhos não seguem a nuvem — dois para baixo, três para cima. Nenhum método sério deve ignorar isso sem pelo menos declarar o que fez.

```
na_bloco <- aggregate(cbind(nitrogen = is.na(campo$nitrogen),
                           organic_matter = is.na(campo$organic_matter)) ~ block,
                     data = campo, FUN = mean)
na_longo <- data.frame(
  block = rep(na_bloco$block, 2),
  variavel = rep(c("nitrogen", "organic_matter"), each = nrow(na_bloco)),
  fracao = c(na_bloco$nitrogen, na_bloco$organic_matter)
)
ggplot(na_longo, aes(block, fracao, fill = variavel)) +
  geom_col(position = "dodge", width = .7) +
  scale_fill_manual(values = c(nitrogen = cores_smf[["primaria"]],
                              organic_matter = cores_smf[["destaque"]])) +
  labs(x = "bloco", y = "fração de ausentes", fill = NULL,
       title = "Missingness pattern by block") +
```

```
theme(legend.position = "top")
```

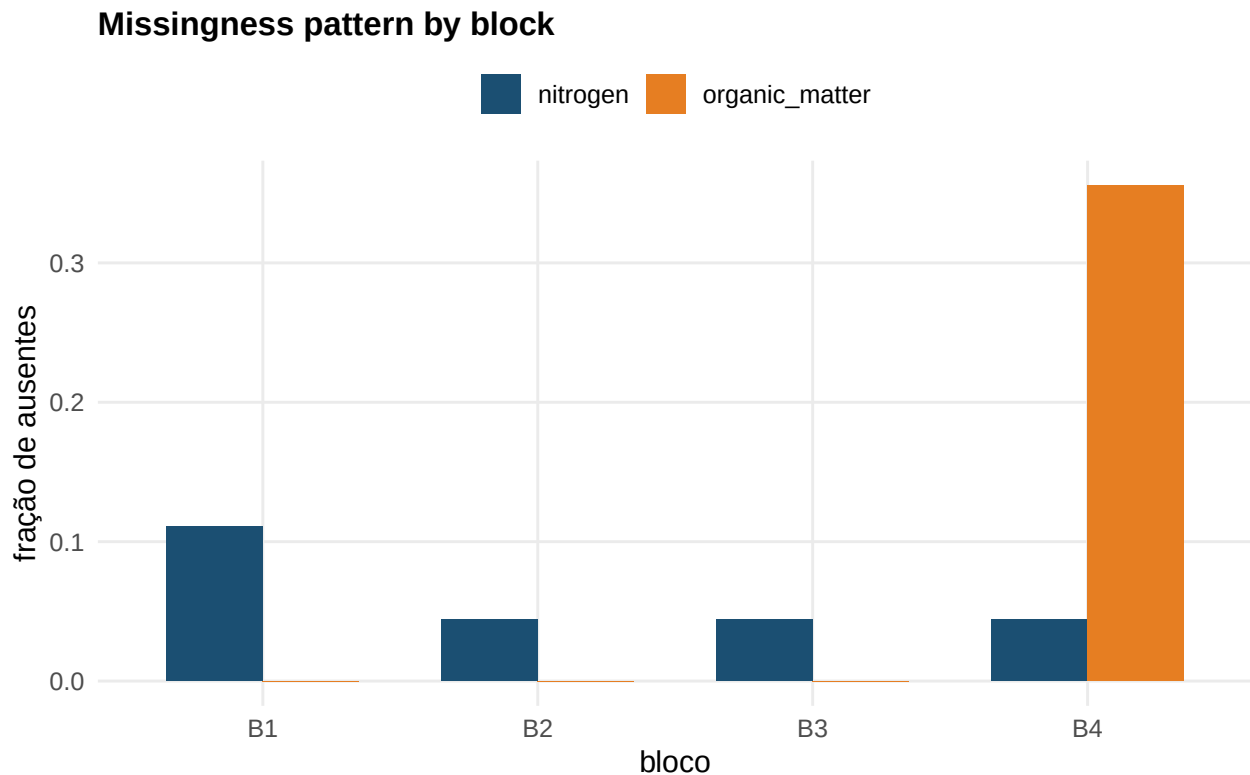


Figura 3.2: Ausências por bloco: o nitrogênio falta de forma aleatória (MCAR), mas a matéria orgânica falta mais no bloco B4 (MAR).

⚠ Armadilha

As duas ausências **parecem** o mesmo problema e não são. Ignorar o padrão MAR (ausência que depende do bloco) enviesa a média da matéria orgânica justamente no bloco que mais falta. Guarde esta figura: ela volta na Parte 2, quando comparamos estratégias de imputação.

3.2 Dois desfechos: classe desbalanceada e contagem superdispersa

```
doenca <- ler("disease_binary")
prev <- mean(doenca$disease)
cat("plantas:", nrow(doenca), "| prevalência observada:",
    fmt(prev, 3), "| rótulos invertidos:", length(verdade$disease_binary$flip_ids), "\n")
```

```
plantas: 400 | prevalência observada: 0,105 | rótulos invertidos: 20
```

```

tab_classe <- data.frame(
  classe = c("sadia (0)", "doente (1)"),
  n = as.integer(table(doenca$disease)),
  fracao = as.numeric(prop.table(table(doenca$disease)))
)
ggplot(tab_classe, aes(classe, n, fill = classe)) +
  geom_col(width = .6, show.legend = FALSE) +
  geom_text(aes(label = paste0(n, " (", fmt(100 * fracao, 1), "%)")), vjust = -0.4) +
  scale_fill_manual(values = c(cores_smf[["primaria"]], cores_smf[["alerta"]])) +
  labs(x = NULL, y = "plantas",
       title = "Imbalanced binary outcome") +
  ylim(0, max(tab_classe$n) * 1.15)

```

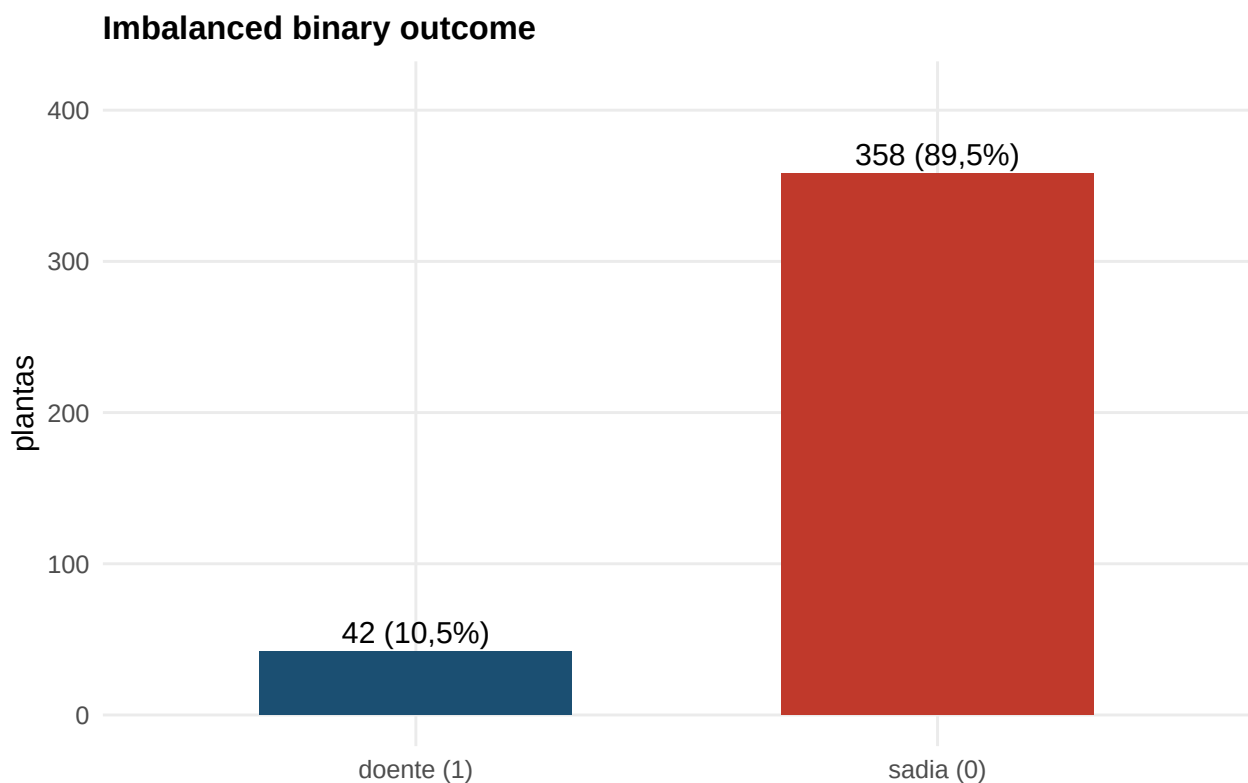


Figura 3.3: Desbalanceamento de classes: apenas cerca de 10% das plantas são positivas, e 5% dos rótulos foram invertidos de propósito.

⚠ Armadilha

Com 10% de positivos, **acurácia é uma métrica enganosa**: prever “sadia” para todas as plantas acerta cerca de 89,5% e não aprende nada. Além disso, 5% dos rótulos foram invertidos: existe um **teto de desempenho** abaixo de 100%. As Partes 3 e 4 mostram como medir e comunicar esse teto.

3.3 Estrutura: grupos, tempo, espaço e alta dimensão

```
fazendas <- ler("farm_groups")
ggplot(fazendas, aes(stats::reorder(farm, yield, FUN = median), yield)) +
  geom_boxplot(fill = cores_smf[["claro"]], outlier.size = .8) +
  labs(x = "fazenda (ordenada pela mediana)", y = "rendimento (kg/ha)",
       title = "Clustered data: farm effects") +
  theme(axis.text.x = element_text(size = 7, angle = 90, vjust = .5))
```

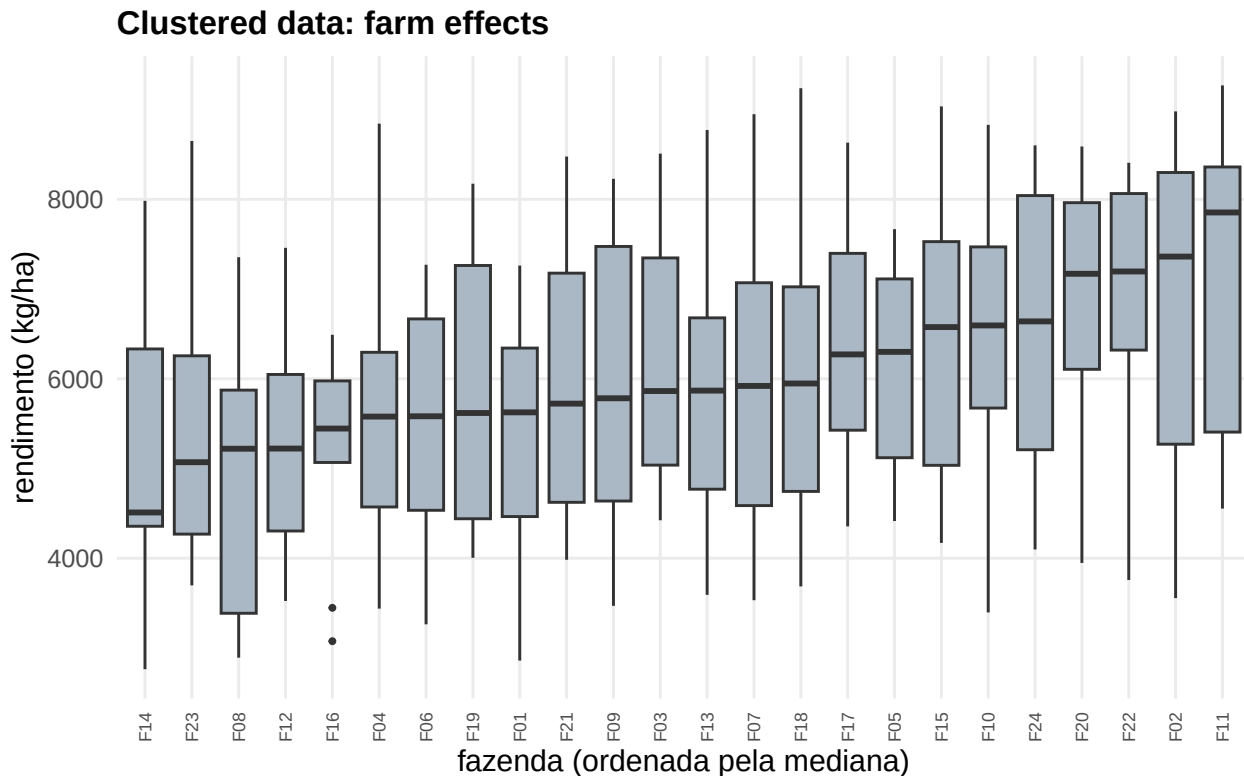


Figura 3.4: Agrupamento: 24 fazendas com 10 talhões cada; o rendimento médio de cada fazenda desloca a nuvem inteira.

```
clima <- ler("climate_series")
clima$choque <- clima$day %in% verdade$climate_series$shock_ids
ggplot(clima, aes(date, temperature)) +
  geom_line(colour = cores_smf[["neutro"]], linewidth = .4) +
  geom_point(data = subset(clima, choque), colour = cores_smf[["alerta"]], size = 2.4) +
  labs(x = NULL, y = "temperatura (°C)",
       title = "Time series: trend, season and shocks") +
  theme(legend.position = "none")
```

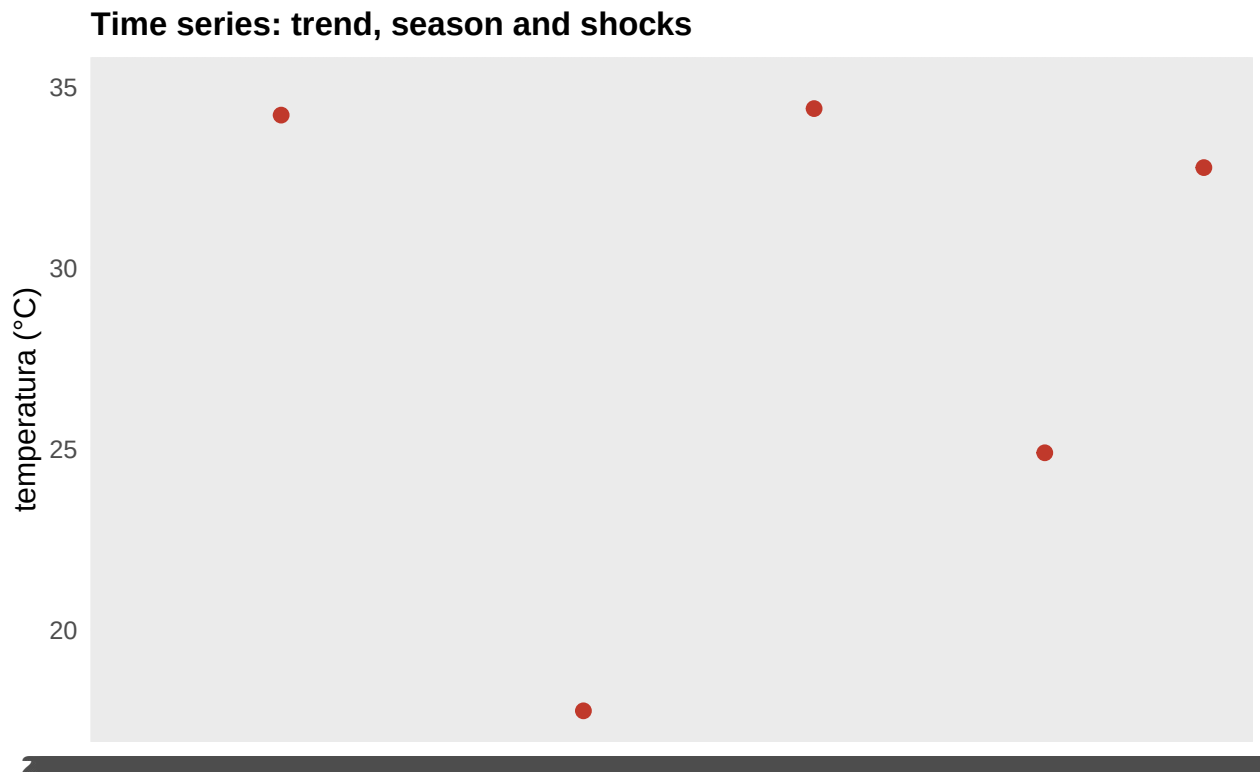


Figura 3.5: Série temporal simulada: tendência, sazonalidade, autocorrelação e cinco choques plantados (vermelho).

```
solo <- ler("soil_spatial")
ggplot(solo, aes(x, y, colour = organic_matter)) +
  geom_point(size = 2.1) +
  scale_colour_gradient(low = "#f6e8c3", high = cores_smf[["primaria"]],
                        name = "MO (%)") +
  coord_equal() +
  labs(x = "x (talhão)", y = "y (talhão)",
       title = "Spatial autocorrelation: organic matter") +
  theme(legend.position = "right")
```

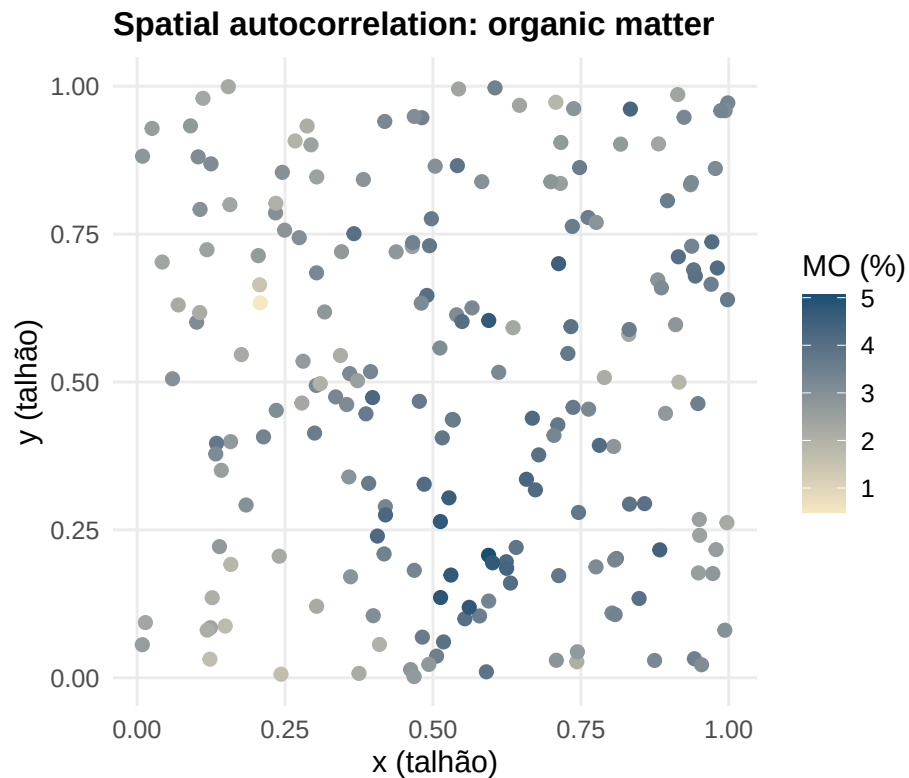


Figura 3.6: Autocorrelação espacial: matéria orgânica em 200 pontos; vizinhos se parecem (alcance 0,25 na escala do talhão).

```
espectro <- ler("spectral_pn")
ondas <- as.numeric(sub("w", "", grep("^w", names(espectro), value = TRUE)))
matriz <- as.matrix(espectro[, grep("^w", names(espectro))])
sel <- c(1:7, 11, 47, 83, 109)
longo <- do.call(rbind, lapply(sel, function(i) {
  data.frame(lambda = ondas, absorbancia = matriz[i, ],
             amostra = espectro$sample_id[i],
             outlier = espectro$sample_id[i] %in% verdade$spectral_pn$outlier_ids)
}))
ggplot(longo, aes(lambda, absorbancia, group = amostra, colour = outlier)) +
  geom_line(alpha = .75, linewidth = .35) +
  scale_colour_manual(values = c(`FALSE` = cores_smf[["claro"]],
                                `TRUE` = cores_smf[["alerta"]]),
                     labels = c("regulares (amostra)", "outliers (4)")) +
  labs(x = "comprimento de onda (nm)", y = "absorbância", colour = NULL,
       title = "High dimension: p >> n") +
  theme(legend.position = "top")
```

High dimension: $p \gg n$

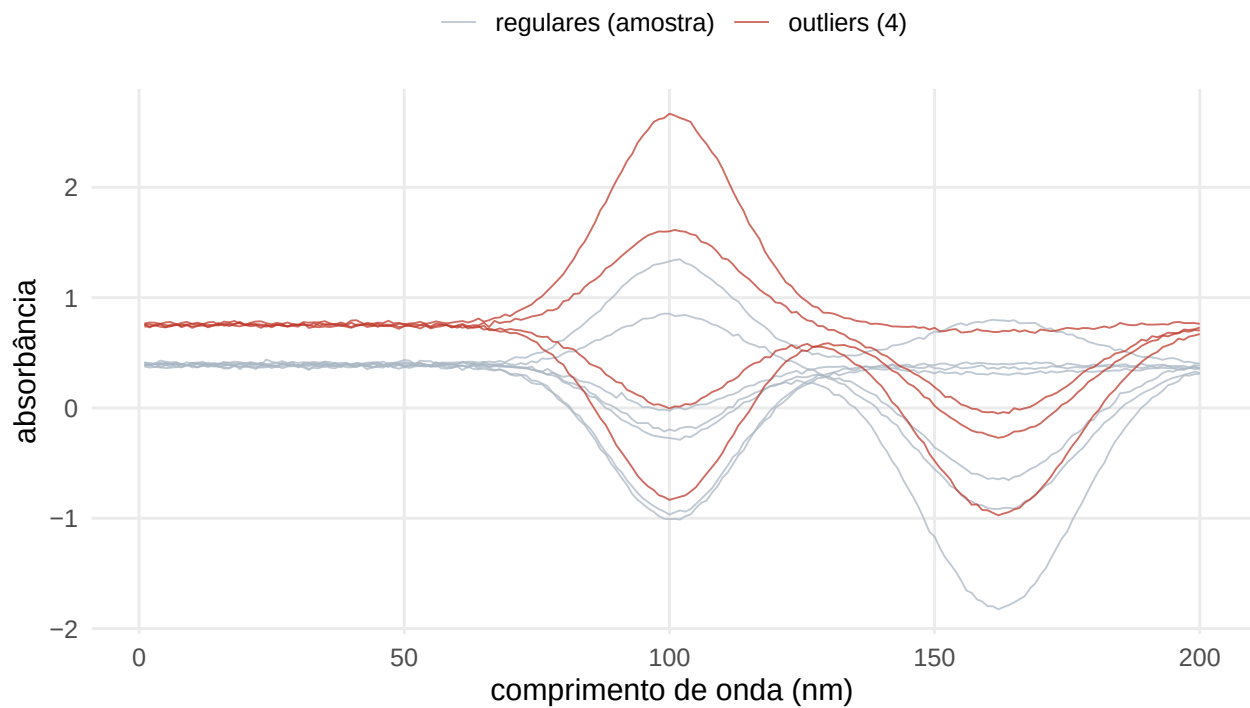


Figura 3.7: Alta dimensão: 200 comprimentos de onda para 120 amostras; quatro espectros tiveram a linha de base deslocada (vermelho).

```
grade <- expand.grid(x1 = seq(-1, 1, by = .04), x2 = seq(-1, 1, by = .04))
grade$y <- sin(2 * pi * grade$x1) + 0.5 * grade$x2^2 + 0.8 * grade$x1 * grade$x2
ggplot(grade, aes(x1, x2, fill = y)) +
  geom_raster() +
  scale_fill_viridis_c(option = "C", name = "y") +
  coord_equal() +
  labs(x = "x1", y = "x2",
       title = "Deep learning target: nonlinear surface") +
  theme(legend.position = "right")
```

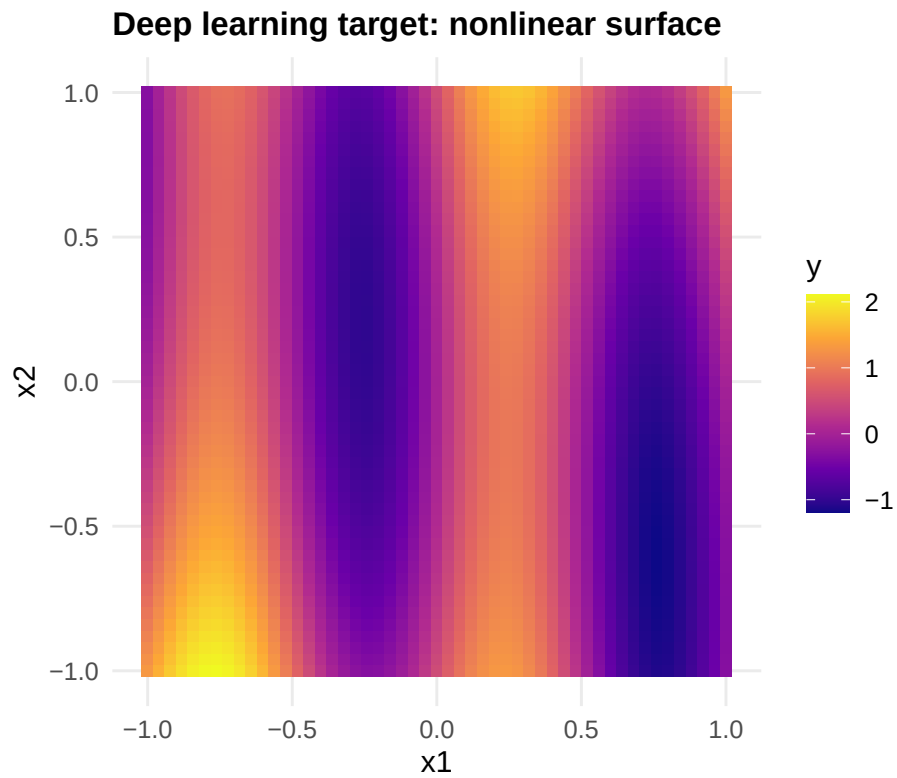


Figura 3.8: Não linearidade plantada para a Parte 6: a superfície verdadeira combina seno, quadrado e interação.

Os conjuntos de contagem (`pest_counts`), de amostra pequena (`bayes_small`) e de classificação entram nas partes correspondentes, com as mesmas regras: tabela de discrepâncias, execução real e comparação com a verdade.

4 Auditoria e contratos

Antes de qualquer modelo, três declarações: **o que** se quer prever, **quais** variáveis entram e **como** as observações se relacionam. No pacote, cada uma é um objeto.

```
tarefa <- smf_task_spec("regression", "yield")
dados_spec <- smf_data_spec(
  target = "yield",
  predictors = c("nitrogen", "potassium", "water", "organic_matter"),
  id_column = "plot_id",
  units = c(yield = "kg/ha", nitrogen = "kg/ha", water = "mm"),
  categorical = c("block", "genotype")
)
desenho_spec <- smf_design_spec(id_column = "plot_id", block_columns = "block")
print(tarefa); print(desenho_spec)
```

```
<sciModelFlowR::TaskSpec>
@ kind      : chr "regression"
@ target    : chr "yield"
@ positive_label: NULL
@ objective  : chr(0)

<sciModelFlowR::DesignSpec>
@ id_column      : chr "plot_id"
@ group_columns  : chr(0)
@ block_columns  : chr "block"
@ repeated_unit  : chr(0)
@ time_column    : chr(0)
@ coordinate_columns : chr(0)
@ external_domain_column: chr(0)
@ hierarchy      : list()
@ experimental_unit : chr(0)
```

A auditoria percorre o conjunto e devolve o que precisa ser visto **antes** de modelar: ausências por variável, duplicatas, cardinalidade, sinais de vazamento e avisos de qualidade.

```
auditoria <- smf_audit_data(campo, dados_spec, desenho_spec)
tab_smf(subset(auditoria@missingness, n_missing > 0),
  caption = "Auditoria: ausências encontradas no ensaio de campo.")
```

Tabela 4.1: Auditoria: ausências encontradas no ensaio de campo.

	variable	n_missing	fraction_missing
4	nitrogen	11	0,061
7	organic_matter	16	0,089

```
cat("duplicatas (linhas):", length(auditoria@duplicates$rows),
    "| flags de qualidade:", length(auditoria@quality_flags),
    "| sinais de vazamento:", length(auditoria@leakage), "\n")
```

duplicatas (linhas): 0 | flags de qualidade: 0 | sinais de vazamento: 0

```
ggplot(auditoria@cardinality, aes(stats::reorder(variable, n_unique), n_unique)) +
  geom_col(fill = cores_smf[["primaria"]], width = .7) +
  geom_text(aes(label = n_unique), hjust = -0.2, size = 3) +
  coord_flip() +
  labs(x = NULL, y = "valores únicos",
       title = "Cardinality audit") +
  ylim(0, max(auditoria@cardinality$n_unique) * 1.12)
```

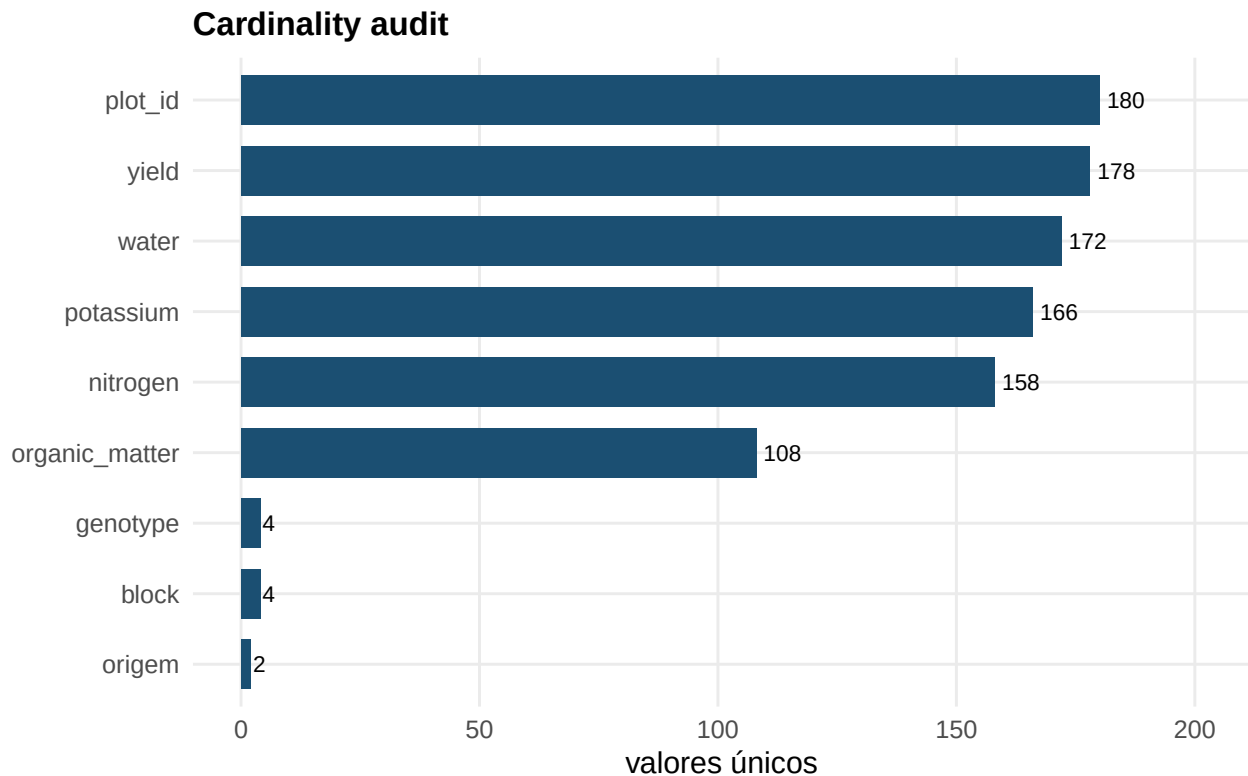


Figura 4.1: Cardinalidade por variável: identificadores únicos, blocos e genótipos repetidos, doses contínuas.

i Interpretação

A auditoria **não** procura outliers: ela cobre estrutura (ausências, duplicatas, cardinalidade, vazamento). Detecção de discrepância numérica é análise, não auditoria — e é o que fazemos a seguir, à mão, para aprender o critério.

4.1 Encontrando os outliers plantados

Um critério simples e robusto é o **z robusto**: distância da mediana dividida pelo desvio absoluto mediano (MAD). A vantagem sobre o z clássico é que o próprio outlier não infla a régua.

```
z_robusto <- (campo$yield - stats::median(campo$yield, na.rm = TRUE)) /
  stats::mad(campo$yield, na.rm = TRUE)
campo$z_robusto <- z_robusto
campo$flag_z <- abs(z_robusto) > 3.5
plantados <- campo$plot_id %in% verdade$field_yield$outlier_ids
confusao <- table(plantado = plantados, flag_z = campo$flag_z)
tab_smf(as.data.frame.matrix(confusao),
  caption = "Detecção por z robusto (> 3,5) contra os outliers plantados.")
```

Tabela 4.2: Detecção por z robusto (> 3,5) contra os outliers plantados.

	FALSE
FALSE	175
TRUE	5

```
ggplot(campo, aes(seq_along(z_robusto), z_robusto)) +
  geom_point(aes(colour = origem), size = 2) +
  geom_hline(yintercept = c(-3.5, 3.5), linetype = "dashed", colour = cores_smf[["neutro"]])
  scale_colour_manual(values = c("parcela regular" = cores_smf[["primaria"]],
    "outlier plantado" = cores_smf[["alerta"]])) +
  labs(x = "posição no arquivo (não é o id)", y = "z robusto do rendimento",
    colour = NULL,
    title = "Robust z-score screening") +
  theme(legend.position = "top")
```

Robust z-score screening

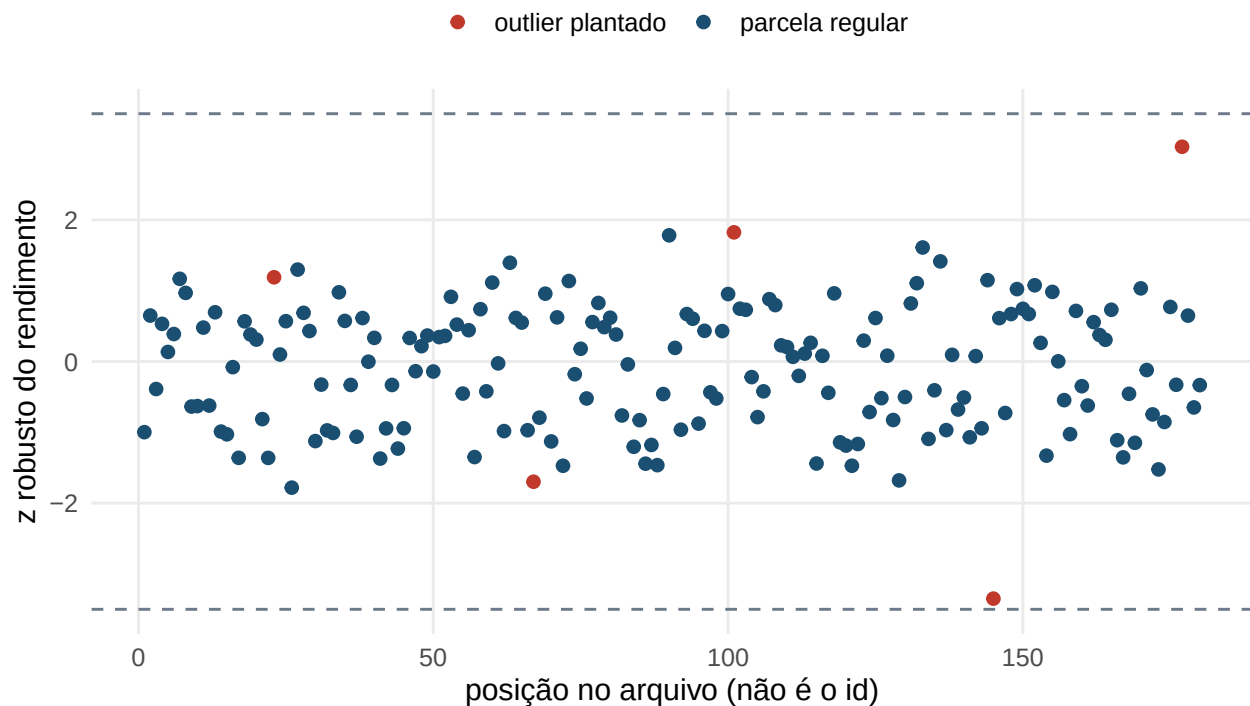


Figura 4.2: Z robusto por parcela: a régua calculada com mediana e MAD separa os cinco pontos plantados do restante.

! Interpretação

A régua encontrou 0 parcelas. Das cinco plantadas, 0 foram detectadas e 0 alarme(s) falso(s) apareceu(aram). Detecção não é julgamento: o passo seguinte é **investigar** (erro de digitação? parcela atípica? tratamento?) e **declarar** a decisão. Na Parte 3 veremos o efeito dessas decisões no ajuste e nos resíduos.

4.2 Contratos: o que o pacote recusa

O pacote transforma regras científicas em erros de execução. Quatro tentativas inválidas, executadas agora:

```
tentar <- function(expr) tryCatch(expr, error = function(e) conditionMessage(e))
r1 <- tentar(smf_data_spec("yield", c("nitrogen", "yield"), "plot_id"))
r2 <- tentar(smf_fit_preprocessor(campo, smf_preprocess_spec("median"),
                                dados_spec, role = "test"))
r3 <- tentar(smf_apply_imbalance(campo, "yield",
                                smf_imbalance_spec("downsample"),
                                context = "test", seed = 20260919))
```

```

r4 <- tentar(smf_bootstrap(fazendas, smf_bootstrap_spec("case", n_resamples = 49L),
  statistic = function(z) c(media = mean(z$yield)),
  design = smf_design_spec(id_column = "plot_id",
    group_columns = "farm")))

recusas <- data.frame(
  tentativa = c("alvo também como preditor",
    "pré-processamento ajustado em role = test",
    "balanceamento aplicado em context = test",
    "bootstrap iid em dados com grupos declarados"),
  resposta = c(r1, r2, r3, r4)
)
names(recusas) <- c("Tentativa inválida", "Mensagem devolvida pelo pacote")
tab_smf(recusas, caption = "Contratos verificados em tempo de execução.")

```

Tabela 4.3: Contratos verificados em tempo de execução.

Tentativa inválida	Mensagem devolvida pelo pacote
alvo também como preditor - @target cannot also be listed in @predictors	object is invalid:
pré-processamento ajustado em role = test	Managed preprocessing can only be fitted on training data.
balanceamento aplicado em context = test	Sampling or class-weight estimation must not be learned/applied from assessment or final test labels.
bootstrap iid em dados com grupos declarados	The requested bootstrap unit does not preserve dependence declared in DesignSpec.

Interpretação

As quatro recusas são a mesma regra em quatro lugares: **informação de avaliação não participa da construção do modelo, e a unidade de reamostragem respeita a dependência declarada**. Um pacote que aceitasse qualquer uma dessas chamadas devolveria números otimistas de aparência normal.

No seu experimento

Comece todo projeto com três perguntas: (1) qual é a unidade experimental e qual é o identificador? (2) existe agrupamento (fazenda, bloco, talhão), tempo ou espaço declarado? (3) quais colunas são “de avaliação” e nunca devem tocar o treino? As respostas viram `smf_data_spec()` e `smf_design_spec()`.

5 Exercícios da Parte 1

Exercício 1.1 — MCAR contra MAR

No ensaio de campo, mostre numericamente que a ausência de **nitrogen** não depende do bloco, enquanto a de **organic_matter** depende. Qual das duas estratégias de imputação você suspeita que sofrerá mais? Guarde a resposta para a Parte 2.

Exercício 1.2 — Outra régua de detecção

Detecte os outliers de **yield** com a regra do intervalo interquartil ($1,5 \times \text{IQR}$ além dos quartis) e compare com o **z** robusto: quem encontra mais plantados, quem erra mais? Por que o IQR é ainda mais frágil que o MAD?

Exercício 1.3 — Auditoria de outro conjunto

Construa `smf_task_spec()`, `smf_data_spec()` e `smf_design_spec()` para o conjunto **disease_binary** (alvo **disease**; preditores **x1** a **x4**; id **plant_id**) e rode `smf_audit_data()`. O que a auditoria mostra sobre a prevalência? Ela “vê” o desbalanceamento? E o ruído de rótulo?

Exercício 1.4 — Por que o bootstrap iid foi recusado?

Explique, em duas frases, o que aconteceria com o intervalo de confiança da média de **yield** em **farm_groups** se cada talhão fosse tratado como independente. (Dica: a variância entre fazendas é 300 e a residual 120.)

Soluções

1.1. A fração de ausência de **nitrogen** por bloco é praticamente constante (MCAR), enquanto **organic_matter** concentra as ausências no bloco B4 (MAR). A imputação pela média global deve sofrer mais no MAR, porque a média do bloco B4 difere das demais; uma imputação que respeite o bloco (ou um modelo que inclua **block**) reduz o viés.

1.2. A regra do IQR tende a marcar mais pontos na cauda longa criada pela heterocedasticidade (rendimentos altos com doses altas), misturando “discrepante” com “esperado”. O MAD é mais estável porque a mediana e o MAD quase não se movem com a contaminação.

1.3. A auditoria devolve a contagem de ausências (zero), duplicatas (zero) e cardinalidade; o desbalanceamento aparece apenas como **cardinalidade** da variável alvo (dois valores únicos) e o ruído de rótulo não é detectável por auditoria — nenhum procedimento automático sabe qual rótulo foi invertido.

1.4. Tratar talhões como independentes ignora a variância entre fazendas (sd 300 contra residual 120), produzindo intervalos **estritos demais** e conclusões confiantes demais. A unidade de reamostragem correta é a fazenda.

```
cat("Parte 1 concluída em", format(Sys.time(), "%Y-%m-%d %H:%M"),  
    "| pacote", as.character(packageVersion("sciModelFlowR")), "\n")
```

Parte 1 concluída em 2026-09-19 00:21 | pacote 1.0.2